

CODEFEST AD ASTRA 2026

Etapla 1 — Construcción de una Base de Conocimiento Vectorial

Documentación técnica · Universidad de los Andes · Fuerza Aeroespacial Colombiana

Repositorio: codefest-ad-astra · Rama: integracion-fase3-4 · Agosto de 2026

1. Resumen ejecutivo

Este documento describe el sistema construido para la Etapa 1: una base de conocimiento vectorial que, ante una consulta en lenguaje natural, devuelve los documentos y fragmentos más relevantes de un corpus multiformato y multilingüe sobre tres fenómenos de interés aeroespacial y territorial. El pipeline extrae y normaliza el texto del corpus (PDF, HTML, JSON, CSV, XLSX, imágenes y mapas PBF), lo fragmenta respetando fronteras de oración, codifica cada fragmento con un encoder multilingüe y lo indexa en FAISS; sobre ese índice se responden las 50 consultas de evaluación. En ninguna etapa de indexación o recuperación interviene un modelo generativo: todo se resuelve con encoders, aritmética vectorial y metadata.

Aspecto	Resultado
Cobertura de extracción del corpus	1.754 de 1.835 recursos (95,6 %)
Fragmentos indexados	202.350
Encoder	BAAI/bge-m3 — 1.024 dimensiones, licencia MIT
Índice	FAISS IndexFlatIP sobre vectores normalizados (coseno exacto)
Consultas respondidas	50 de 50 (q001–q050)
Conformidad del archivo de resultados	100 % del esquema exigido, verificado programáticamente
Idiomas cubiertos	Español, inglés y portugués

2. Problema, entregables y restricciones

El reto pide una arquitectura de análisis estratégico alimentada exclusivamente por fuentes abiertas, articulada en tres niveles (global, regional y nacional) sobre tres fenómenos: (1) inteligencia artificial e innovación en entornos militares; (2) seguridad espacial y congestión de la órbita baja (LEO), vista a través de productos analíticos y no de datos orbitales primarios; y (3) dinámicas territoriales en América Latina. La Etapa 1 construye la base vectorial que servirá de infraestructura de recuperación al asistente conversacional de la Etapa 2, y se evalúa exclusivamente por la calidad de la recuperación.

Los entregables son cuatro, empaquetados en un único directorio: la base vectorial (índice FAISS serializado con la función estándar de la librería más un almacén de metadata en JSON Lines, en una subcarpeta por encoder); un archivo de resultados de exactamente 50 líneas, cada una con los 3 documentos y los 10 fragmentos más relevantes; este documento técnico en PDF de máximo 8 páginas; y un script que, a partir del índice y del archivo de consultas, reproduzca el archivo de resultados entregado.

Cuatro reglas no admitían interpretación y determinaron el diseño:

Restricción	Implicación práctica
Prohibido usar modelos generativos (decoders tipo GPT, LLaMA, Claude) en indexación o recuperación	Nada de reranking con LLM, reformulación de consultas ni síntesis. Solo encoders, similitud y metadata.
Ningún fragmento puede contener oraciones incompletas	La fragmentación corta solo en frontera de oración, retrocediendo a la última oración completa que quepa en el presupuesto de tokens.
El orden de las líneas de metadata debe coincidir con los ids internos que FAISS asigna	Una desalineación devuelve textos que no corresponden a los vectores recuperados, sin producir error visible.

El script de generación debe reproducir los resultados entregados	Si no se reproducen, la entrega se excluye. Obliga a un índice exacto y determinista y a versionar el código que tocó los datos.
---	--

La evaluación usa 50 consultas en los tres idiomas, con una lista de relevancia manual no pública durante el reto. Métricas: NDCG@10 sobre los fragmentos (penaliza más los errores en las primeras posiciones) y F1@3 sobre los documentos (sin orden), combinadas en un leaderboard por Conteo de Borda. La relevancia se juzga sobre el contenido textual del fragmento y sobre el archivo original del documento, no sobre los identificadores internos de cada equipo, que cumplen función de trazabilidad y no de emparejamiento.

3. Diseño y pipeline

3.1 Decisiones tomadas antes de escribir código

Decisión	Opción elegida	Por qué
Encoder	Uno solo, multilingüe: BAAI/bge-m3	El corpus mezcla español, inglés y portugués y una consulta en un idioma debe recuperar documentos en otro. Multilingüe nativo, buen desempeño en recuperación densa, licencia MIT y contexto amplio.
Índice FAISS	IndexFlatIP con vectores normalizados	Con norma unitaria el producto interno es exactamente la similitud coseno. Búsqueda exacta y determinista; un índice aproximado introduciría variabilidad entre corridas y comprometería la reproducibilidad exigida.
Chunking	Híbrido: estructural con corte en frontera de oración	Empaqueta párrafos completos dentro de un presupuesto de tokens y solo baja a oración cuando un párrafo no cabe. Cumple la completitud lingüística sin romper la cohesión temática.
Grafo de conocimiento	No implementado	Componente bonus, condicionado a terminar el pipeline principal con margen de tiempo.

El trabajo se dividió en un frente de datos (extracción, limpieza y fragmentación) y un frente de modelo (encoder, embeddings, índice, recuperación y salida). Para no bloquear al segundo, se trabajó contra fixtures sintéticos y un encoder de pruebas determinista, de modo que la integración fuera un acople: cuando llegaron los fragmentos reales, la indexación funcionó sin cambios de diseño.

3.2 Fases 1–2 — Extracción y limpieza

Cada formato exigió su propio extractor, porque el criterio de qué es texto útil cambia entre un PDF y un archivo de mapas:

- PDF: página a página preservando el orden de lectura, para detectar y eliminar encabezados y pies repetidos.
- HTML: eliminación de marcado conservando solo el texto visible.
- JSON: selección explícita de los campos de contenido; url, fecha y autores pasan a metadata en vez de mezclarse con el cuerpo.
- CSV y XLSX: cada fila se convierte en pares columna–valor, de modo que cada dato conserve el nombre de su columna como contexto.
- Imágenes: OCR sobre las que contienen texto relevante, como infografías.
- PBF (mapas por teselas): capas y elementos leídos como pares atributo–valor, conservando un solo nivel de zoom para no duplicar la misma información geográfica de la pirámide.

Sobre el texto extraído se aplica normalización a UTF-8, eliminación de caracteres de control y espacios redundantes, detección del idioma predominante y remoción de boilerplate. Cada documento recibe un

identificador determinista derivado de su ruta relativa, estable entre corridas. Una capa de diagnóstico clasifica cada PDF e imagen problemática y produce los reportes de cobertura de la Sección 6.

3.3 Fase 3 — Fragmentación

El algoritmo empaqueta párrafos completos mientras quepan en el presupuesto de tokens y, cuando un párrafo no cabe, baja a nivel de oración con un divisor multilingüe que distingue un punto de cierre de una abreviatura o un separador decimal.

Parámetro	Valor	Razón
Máximo de tokens por fragmento	450	Deja margen frente al límite de entrada del encoder (512 tokens).
Máximo de palabras por fragmento	250	Es el límite que el formato de salida impone; respetarlo desde el chunking evita recortar después.
Oraciones de solapamiento	1	Reduce el riesgo de que una idea quede partida entre dos fragmentos consecutivos.

Los formatos tabulares obligaron a un tratamiento aparte: una fila de CSV convertida a pares columna–valor no tiene puntuación de cierre, así que el divisor la ve como una única oración gigantesca que excede el presupuesto. Para esos casos hay un fallback que agrupa pares clave–valor conservando claves enteras y, como último recurso, divide por caracteres. Se aplica exclusivamente a formatos estructurados —en prosa una oración larga nunca se parte, que es el requisito duro del reto— y cada uso queda registrado con su motivo en un archivo de advertencias, para poder auditarlo.

3.4 Fase 4 — Embeddings e índice vectorial

Cada fragmento se convierte en un vector de 1.024 dimensiones normalizado a norma unitaria, y los vectores se insertan en el índice en el mismo orden en que se leyeron los fragmentos; la metadata se escribe en paralelo, línea a línea, en ese orden. Tres decisiones de implementación protegen la validez de la entrega:

- Verificación de la normalización: la construcción rechaza vectores cuya norma se desvíe de 1 más allá de una tolerancia mínima, en lugar de indexarlos en silencio. Un índice sin normalizar produce rankings mal ordenados sin que nada lo delate.
- Escritura atómica y validación de alineación: la correspondencia entre el id interno de FAISS y la línea de metadata se valida antes de escribir y al cargar; los archivos se escriben en temporales y se reemplazan al final, de modo que una corrida interrumpida no deje un índice completo junto a una metadata a medias.
- Codificación por bloques con reanudación: se guardan checkpoints cada bloque y el proceso se reanuda sin recodificar lo hecho. El checkpoint almacena una firma del archivo de entrada, el modelo y la dimensión, y se niega a reanudar si algo cambió: reanudar sobre otro archivo mezclaría vectores de dos corpus y produciría la desalineación silenciosa que el resto del diseño evita.

Los embeddings se generaron en GPU: la medición local en CPU dio 0,83 fragmentos por segundo con fragmentos de 512 tokens, decenas de horas para el volumen final del corpus. Dos utilidades adicionales, que reutilizan el mismo encoder y las mismas garantías de alineación, permiten añadir fragmentos a un índice ya construido sin recodificar los existentes y recodificar un subconjunto cuando cambia la forma de construir el texto que ve el encoder.

3.5 Fases 6–7 — Recuperación y generación de resultados

La consulta se codifica con el mismo encoder que construyó el índice, se normaliza y se compara contra él; para cada id devuelto se recupera su metadata. El punto delicado —usar el mismo encoder y los

misimos prefijos de instrucción que en la indexación— se resolvió persistiendo modelo, dimensión y prefijos en un manifiesto junto al índice: el buscador los lee de ahí en lugar de confiar en que quien consulta los configure bien, lo que elimina una clase entera de errores silenciosos. La agregación a nivel de documento suma las puntuaciones de todos los fragmentos recuperados de cada documento y toma los tres primeros; sumar premia al documento que aporta varios fragmentos relevantes, no solo uno muy bueno.

El generador recorre las 50 consultas y produce los 3 documentos y los 10 fragmentos exigidos, con dos detalles que resuelven casos borde reales: búsqueda adaptativa —se recuperan 60 fragmentos candidatos y se amplía el número si los 3 documentos seleccionados no aportan 10 fragmentos entre todos— y recorte defensivo a 250 palabras cortando en frontera de oración. Antes de terminar valida su propia salida (número de líneas, documentos y fragmentos por línea, límite de palabras) y falla explícitamente si algo no cuadra: es preferible fallar al generar que descubrirlo cuando el evaluador rechaza el archivo.

4. Estructura del código y archivos clave

El proyecto es un paquete de Python gestionado con uv y organizado por fase del pipeline; las dependencias apuntan siempre en el sentido del flujo: la recuperación conoce a la indexación, pero la indexación no sabe nada de la recuperación.

```
codefest-ad-astra/
├── src/codefest_ad_astra/
│   ├── ingest/    Fases 1-3: extractors, cleaning, extraer_y_limpiar, pipeline, chunking,
│   │               validation, quality_metrics, pdf_validator, ocr_validator, diagnostics
│   ├── indexing/  Fase 4: encoders, faiss_store, build_index, extend_index,
│   │               reencode_subset, search, verificar
│   └── retrieval/ Fases 6-7: recuperar (agregación a documento), generador (resultados.jsonl)
├── tests/         data/ (consultas.jsonl, diagnostics/) docs/ README.md pyproject.toml
└── entrega/      resultados.jsonl + base_vectorial/ (index.faiss + metadata.jsonl, no versionado)
```

El repositorio versiona el código que funciona, no los datos: el corpus crudo, los fragmentos intermedios y el índice son artefactos pesados y regenerables que quedan fuera del control de versiones, y deben transferirse por otro medio a quien arme el paquete final. De los cerca de treinta módulos, siete concentran las decisiones que determinan la calidad del resultado:

Archivo	Qué resuelve y por qué importa
ingest/extraer_y_limpiar.py	Punto de entrada único: dada la ruta de cualquier archivo del corpus devuelve texto limpio, idioma y metadata. Aísla siete formatos detrás de una sola llamada.
ingest/chunking.py	Fragmentación con corte en frontera de oración, solapamiento y fallback tabular. Es donde se cumple —o se incumple— la completitud lingüística.
indexing/encoders.py	Normalización a norma unitaria y prefijos de instrucción según la familia del modelo. Omitir los prefijos degrada la recuperación sin avisar.
indexing/faiss_store.py	Índice, persistencia atómica y validación índice↔metadata. Custodia la invariante de la que depende la entrega: la línea <i>i</i> describe el vector <i>i</i> .
indexing/build_index.py	Orquesta la Fase 4 (lectura, validación, codificación por bloques, checkpoints, manifiesto). Es el proceso más largo y produce el entregable obligatorio.
indexing/search.py	Búsqueda top-k por coseno leyendo del manifiesto el modelo y los prefijos: garantiza que consulta y documentos vivan en el mismo espacio semántico.
retrieval/generador.py	Produce y valida el archivo de resultados. Es el entregable que debe reproducirse para que la entrega sea evaluada.

5. Ejecución del pipeline

Cuatro comandos, cada uno consumiendo la salida del anterior; todos reanudables o idempotentes, de modo que una interrupción no obliga a empezar de cero.

- 1) `uv run python -m codefest_ad_astra.ingest.pipeline --corpus data/raw \`
`--salida data/processed/documentos.jsonl`
- 2) `uv run python -m codefest_ad_astra.ingest.chunking --entrada data/processed/documentos.jsonl \`
`--salida data/processed/chunks.jsonl --errores data/processed/chunking_errors.jsonl \`
`--tokenizer-model BAAI/bge-m3`
- 3) `uv run python -m codefest_ad_astra.indexing.build_index --entrada data/processed/chunks.jsonl \`
`--salida entrega/base_vectorial --modelo BAAI/bge-m3 --device cuda`
- 4) `uv run python -m codefest_ad_astra.retrieval.generador --base entrega/base_vectorial/encoder_bge-m3 \`
`--consultas data/consultas.jsonl --salida entrega/resultados.jsonl`

Un comando adicional, `uv run python -m codefest_ad_astra.indexing.verificar --base entrega/base_vectorial/encoder_bge-m3`, revisa una base ya construida: alineación índice/metadata, normalización de los vectores, campos obligatorios y distribución del contenido por fenómeno, idioma y formato.

6. Resultados

De los 1.835 recursos del corpus, 1.754 se extrajeron y limpiaron correctamente (95,6 %). Los 81 restantes están clasificados y explicados, no son pérdidas desconocidas:

Categoría	Cant.	Causa	¿Recuperable?
Manifiestos y catálogos	8	Índices del proceso de descarga, no contenido documental	No aplica
PDFs escaneados sin OCR	~45	Concentrados casi todos en una misma fuente	Sí, aplicando OCR
PDFs corruptos	2	Archivo de origen dañado	No, sin el original
Imágenes sin texto	4	Fotografías, no infografías	No aplica
Contenido genuinamente corto	~15	Boletines y artículos breves	No: es el contenido real

El índice final contiene 202.350 fragmentos vectorizados con bge-m3 en 1.024 dimensiones sobre un índice plano de producto interno con vectores normalizados, lo que hace la búsqueda exacta y equivalente a similitud coseno. Cada fragmento conserva en la metadata los ocho campos obligatorios más campos de trazabilidad hacia el texto original: idioma, número de palabras y posición de inicio y fin en el documento fuente.

El archivo de resultados se verificó programáticamente contra el esquema exigido y todas las comprobaciones pasaron:

Comprobación	Exigido	Obtenido
Número de líneas / identificadores	50, q001...q050	50, correcto
Documentos y fragmentos por consulta	3 y 10	3 y 10 en las 50 consultas
Palabras por fragmento	máximo 250	máx. 250; media 214,5; mín. 65 (0 exceden)
Campos de cada objeto	rank, doc_id / rank, chunk_id, doc_id, text	Correctos
Fragmentos repetidos dentro de una consulta	ninguno	ninguno

Las 50 consultas produjeron 500 fragmentos que citan 76 documentos distintos: esa dispersión sugiere que el sistema no colapsa sobre un puñado de documentos dominantes, síntoma habitual de un índice mal

normalizado o de una agregación mal calibrada. NDCG@10 y F1@3 no pueden calcularse por cuenta propia porque la lista de relevancia de referencia no es pública durante el reto; lo que aquí se reporta es conformidad de formato y consistencia interna, no calidad de recuperación medida contra el estándar de oro.

La suite ejecuta 60 pruebas: 58 pasan y 2 fallan por el arnés de pruebas en Windows (una invoca el intérprete por su ruta de Linux y otra depende de archivos temporales POSIX), no por el pipeline. La cobertura de pruebas es el punto más débil del proyecto: durante la integración de ramas se perdió una suite de 82 pruebas del módulo de indexación —alineación índice/metadata, normalización, checkpoints, reanudación y búsqueda— y quedó en su lugar una única prueba de extremo a extremo. Ese código sigue en producción, pero ya no está protegido frente a regresiones.

7. Asuntos abiertos antes de cerrar la entrega

- Carpeta de entrega incompleta. Hoy solo contiene el archivo de resultados. Faltan: el script de generación en la raíz con el nombre exigido (hoy vive dentro del paquete de código; basta un archivo puente que invoque al módulo), este documento en PDF y la base vectorial —índice y metadata—, que al estar excluida del control de versiones debe copiarse desde la máquina donde se construyó. La reproducción exigida no depende de reconstruir el índice: el script lee el índice entregado, que es en sí mismo un entregable binario, y sobre un índice plano la búsqueda es determinista.
- El preprocesamiento del texto vectorizado no está versionado. Antes de vectorizar se retiran las URL del fragmento, para que el embedding represente el contenido y no el ruido de una dirección; el texto de la metadata las conserva. Esa transformación se aplicó desde una versión local del código de indexación que no se registró. No compromete la evaluación —el índice viaja construido y la transformación solo afectaba al lado de los pasajes, no al de las consultas— pero impide reconstruir el índice desde el repositorio con vectores idénticos, que es lo necesario para auditar, reindexar tras una corrección del chunking o retomar en la Etapa 2. Registrar esa función cierra la brecha.
- Documentación del repositorio desactualizada. El README describe como pendientes las fases de recuperación y generación, ya implementadas, y documenta un paquete eliminado durante la integración.
- Limitación conocida en fragmentos tabulares. El fallback se activa al detectar ausencia total de puntuación de cierre; una fila con un valor decimal incluye un punto, así que no se activa y esa fila se emite como fragmento sobredimensionado. No rompe la completitud lingüística, pero reduce la recuperabilidad de ese subconjunto. La corrección es reutilizar ahí la lógica que ya distingue puntos decimales de puntos de cierre.
- Trabajo opcional no abordado. No se implementó el grafo de conocimiento (puntuación adicional) ni un segundo índice con encoder complementario, aunque la arquitectura ya lo soporta y la fusión por Reciprocal Rank Fusion sería una extensión acotada.

8. Conclusión

El proyecto cumple el objetivo central de la Etapa 1: una base de conocimiento vectorial sobre el 95,6 % del corpus, con 202.350 fragmentos indexados en un espacio semántico multilingüe, y un módulo de recuperación que responde las 50 consultas produciendo un archivo de resultados conforme al esquema en todas sus verificaciones. Las decisiones técnicas se subordinaron a las restricciones del reto antes que a la sofisticación: índice plano y exacto en vez de aproximado porque la reproducibilidad era requisito de exclusión; un solo encoder multilingüe robusto en vez de varios especializados porque la consulta debía cruzar idiomas; y una fragmentación diseñada alrededor de la completitud lingüística en vez del tamaño

óptimo de chunk. En un reto donde el formato incorrecto descalifica, esa subordinación fue la decisión más rentable.

El mayor acierto de proceso fue desacoplar los dos frentes con fixtures sintéticos y un encoder de pruebas determinista, lo que permitió tener la indexación probada antes de que existieran los fragmentos reales. El aprendizaje apunta en dirección contraria: los dos asuntos abiertos —la función de preprocesamiento sin versionar y la suite de pruebas perdida al integrar ramas— no son fallos técnicos sino de disciplina de integración. Versionar solo el código y no los datos es razonable; el matiz es que el código que tocó los datos entregados sí forma parte de lo que hay que conservar. En un pipeline donde una desalineación entre índice y metadata no produce ningún error visible, el control de versiones y las pruebas son lo que distingue un resultado válido de uno que solo lo parece. Con el índice reconstruido desde código versionado y la carpeta de entrega completa, el trabajo queda listo para ser evaluado y para servir de infraestructura de recuperación en la Etapa 2.